

Spec-2: Task Statement

AI Assisted Development Process Course

Individual Spec Task 2 — Extended High-Level Specification & Architecture Spine

1. Executive Summary & Context

In **Spec Task 1**, you formulated the initial high-level specification for the **University AI Compute Management Platform or your own project**, defining the foundational system intent across seven core sections. In **Workshop 2**, you received a rigorous conducted peer evaluation of your specification using the 8-dimension requirement-quality rubric (Clarity, Scope Control, Verifiability, Atomicity, Consistency, Implementation Independence, Traceability, and Decomposition Readiness). Subsequently, in our **Class Exercise**, you explored the **BMAD Whole-system Design System (WDS)** methodology to understand how high-level intent transitions into machine-actionable specification contracts and architectural spines.

Now, in **Spec Task 2**, you will produce the **Extended High-Level Specification & Architecture Spine** for the University AI Compute Management Platform or your own project.

This deliverable represents the **definitive pre-implementation contract**: a complete, handoff-ready design package that bridges business and academic intent with architectural invariants. Following the principles of **Spec-Driven Development (SDD)**, this artifact bundle must be sufficiently rigorous that an AI development agent (e.g., BMAD dev agent) or an independent engineering team can execute implementation with zero ambiguity.

2. Objective

Improve your original `spec.md` using the feedback from peer review and the BMAD **WDS process: What, Decisions, and Specifications**. Your goal is to turn the initial project statement into a more implementation-ready specification while preserving clear boundaries and intentional scope.

Specific aspects to cover:

1. **Remediate & Elevate Spec 1**: Ingest peer-review feedback, resolve ambiguities, eliminate implementation leakage from the spec kernel, and ensure every requirement conforms strictly to **Spec Law**.
2. **Execute the BMAD / WDS Spec Pipeline**: Systematically apply the WDS steps (`wds-0-project-setup`, `wds-1-project-brief`, `bmad-spec`, and `bmad-`

architecture) to synthesize stakeholder needs, formal contracts, and structural invariants.

3. **Define the Architecture Spine:** Formulate the non-negotiable architectural rules, system boundaries, state mutation invariants, technology choices, and numbered Architecture Decisions (ADRs) that prevent downstream implementation drift.
4. **Pass the “Handoff Test”:** Deliver an artifact suite that enables an autonomous AI coding agent or human engineer to begin story generation and code implementation without requiring clarifying questions regarding scope, behavior, or invariants.
5. **Practice Level-3 AI Collaboration:** Leverage AI agents methodically across distinct context windows, documenting decisions, trade-offs, agent pushback, and rejected alternatives in an append-only audit trail.

3. Problem Scope: University AI Compute Management Platform

Your specification and architecture must fully address the institutional problem defined in the project brief:

- **Hardware Fleet Reality:** Heterogeneous local GPU fleet comprising multiple standard compute nodes (e.g., 24 GB VRAM GPUs) and high-capacity multi-GPU nodes (e.g., 2× 48 GB VRAM GPUs) managed via Kubernetes GPU device plugins.
- **Workload Diversity:** Concurrent demands spanning interactive LLM inference (teaching/experimentation), scheduled academic classroom exclusive sessions, and long-running distributed model training jobs (via Kubeflow Training Operator).
- **Governance & Quotas:** Fine-grained access control, fair-share scheduling, quota enforcement based on academic context (course, research group, role), token-level metering (prompt/completion counters), and cost/usage observability.
- **Operational Integrity:** Cluster observability, machine health state management (idle, busy, reserved, failing), automated preemption/re-queuing rules, and conflict avoidance between live classroom reservations and background research jobs.

4. Required Deliverables & Repository Structure

All work must be authored within your repository following the standard BMAD / WDS output structure. Your submission must contain the following discrete artifacts:

```
├── design-artifacts/
│   └── A-Product-Brief/
│       └── project-brief.md          # Product vision, proto-personas,
core concept, metrics
├── bmad-output/
│   └── specs/
│       └── spec-ai-compute-platform/
│           ├── SPEC.md              # The Formal Contract (The "WHAT")
│           └── ARCHITECTURE-SPINE.md # The Architectural Invariants (The
"HOW")
```

```

|          |— .memlog.md          # Append-only decision & prompt audit
trail
|          |— companion-files/
|          |   |— glossary.md      # Domain terminology and entity
definitions
|          |— persona-archetypes.md # Detailed stakeholder profiles &
tension models
|          |   |— diagrams/
|          |   |   |— system-context.mmd # Mermaid C4 context /
structural diagram
|          |   |— state-transitions.mmd # Workload & reservation state
machine
|— peer-review-remediation.md      # Traceability matrix resolving
Workshop 2 feedback

```

5. Detailed Artifact Requirements

Deliverable A: Product Brief (design-artifacts/A-Product-Brief/project-brief.md)

Generated via `wds-1-project-brief` (Saga agent), this artifact captures the strategic “Why”:

- * **Vision & Academic Purpose:** The core mandate of governing institutional AI hardware.
 - * **Stakeholder Personas:** Academic administrators, course instructors, students, lab operators/infrastructure engineers, and privileged AI researchers.
 - * **Value Proposition & Differentiation:** Why centralized governance replaces manual coordination and ad-hoc desktop access.
 - * **High-Level Success Metrics & Constraints:** Quantitative KPIs (utilization rates, zero class disruptions, fair quota distribution).
-

Deliverable B: Formal Specification Contract (SPEC.md)

Distilled via `bmad-spec`, this artifact establishes the **strict behavioral contract** of the system. It defines **WHAT** the system does, not **HOW** it is coded.

Spec Law Compliance:

1. Every capability **MUST** have a unique identifier (CAP-x), an explicit **intent** (observable behavior, WHAT not HOW), and a concrete, measurable **success signal** (binary or threshold-based acceptance test).
2. Non-goals **MUST** be explicit to prevent agent hallucination and scope drift.
3. Constraints **MUST** represent real technical, resource, or institutional boundaries that bend design decisions.

Required Sections in SPEC.md:

1. **Why:** One concise paragraph establishing the institutional problem, affected beneficiaries, and the operational value of solving it. Adapt to your project if applicable.
2. **Capabilities (CAP-1 to CAP-n):** Comprehensive enumeration of observable system capabilities, structured as: * CAP-1: Multi-Tier Quota & Token Metering Engine (Intent + Success Signal) * CAP-2: Classroom Reservation & GPU Lockout Protection (Intent + Success Signal) * CAP-3: Heterogeneous Hardware Placement & Schedulability (Intent + Success Signal) * CAP-4: Distributed Training Workload Lifecycle via Kubeflow (Intent + Success Signal) * CAP-5: Cluster Telemetry, Health State, and Node Eviction (Intent + Success Signal) * CAP-6: Role-Based Access Governance & Academic Group Scoping (Intent + Success Signal) * *(Include additional capabilities identified during project discovery)*.
3. **Constraints:** Real boundaries (e.g., Kubernetes device plugin limitations, VRAM hardware thresholds, on-premises network boundaries, data privacy regulations).
4. **Non-Goals:** Explicit exclusions (e.g., public cloud billing gateways, custom GPU hardware manufacturing, end-user client model authoring IDEs, unmanaged direct SSH access to raw GPU nodes).
5. **Success Signal:** Holistic, measurable criteria verifying platform operational readiness (e.g., *100 concurrent inference streams throttled accurately within 5% quota margin, zero scheduled class preemption failures over a 14-day simulated academic cycle*).
6. **Assumptions:** Exhaustive list of environment, infrastructure, and user behavior assumptions, each explicitly tagged as **[Safe]** or **[Risky]** with impact analysis.

Deliverable C: Architecture Spine (ARCHITECTURE-SPINE.md)

Generated via bmad-architecture, this artifact establishes the **invariants that prevent independent builders from diverging**. It defines the structural **HOW**.

Required Sections in ARCHITECTURE-SPINE.md:

1. **Design Paradigm:** Named architectural pattern (e.g., Event-Driven Microservices, Hexagonal / Ports-and-Adapters, Layered Control Plane) with technical rationale.
2. **System Boundaries & Dependencies:** Component ownership boundaries (e.g., API Gateway / Auth Proxy, Quota & Token Metering Service, Reservation Coordinator, Kubernetes Scheduler/Kubeflow Bridge, Telemetry Ingestion Engine). Who communicates with whom and through what protocol (gRPC, REST, WebSocket, Event Bus).

3. **State Mutation & Invariant Rules:** Single source of truth rules for GPU allocation, reservation collision resolution, token bucket decrement concurrency, and cluster state synchronization.

4. **Technology Stack Decisions:** Definitive language, framework, database, and message broker selections paired with concrete justifications.

5. **Numbered Architecture Decisions (AD-1 to AD-n):** Formal Architecture Decision Records (ADRs) capturing: * Context & Problem * Decision Taken * Status (Accepted / Superseded) * Consequences & Trade-offs (what is gained vs. what is sacrificed) * Rejected Alternatives (why other approaches were dismissed)

6. **Explicit Deferred Items:** Architectural choices intentionally left open for downstream sprint planning.

Deliverable D: Companion Files & Diagrams (companion-files/)

- **glossary.md:** Canonical domain terms (e.g., *VRAM Allocation Unit*, *Token Bucket Quota*, *Preemptible Priority Tier*, *Exclusive Lockout Window*, *Device Plugin Exporter*).
- **persona-archetypes.md:** Deep dive into user motivations, friction points, and permission boundaries.
- **diagrams/:** Renderable Mermaid diagrams covering:
 - System Context & Component Architecture (Mermaid C4 or component diagram).
 - Workload & Reservation State Machine (Mermaid state diagram showing transitions between *Requested* -> *Scheduled* -> *Active* -> *Preempted* -> *Completed* / *Failed*).

Deliverable E: Peer-Review Remediation Matrix (peer-review-remediation.md)

A dedicated traceability document demonstrating how you addressed the feedback received during Workshop 2:

Dimension	Issue / Finding from Workshop 2	Root Cause in Spec 1	Action Taken & Specific Section / ID in Spec 2
Clarity	<i>e.g., Ambiguous wording in quota enforcement</i>	<i>Used “fair allocation” without definition</i>	<i>Created CAP-1 with exact token counter metrics & testable threshold</i>
Impl. Independence	<i>e.g., Premature Redis locking mentioned in Spec 1</i>	<i>Leaked implementation into capability</i>	<i>Moved Redis caching to AD-3 in ARCHITECTURE-SPINE.md; generalized CAP-2</i>
Scope Control	<i>e.g., Unclear whether public billing is supported</i>	<i>Omitted non-goal</i>	<i>Added explicit Non-Goal 3: No external fintech/payment gateway</i>

Dimension	Issue / Finding from Workshop 2	Root Cause in Spec 1	Action Taken & Specific Section / ID in Spec 2
(...)	(Address all 8 workshop dimensions)		

Deliverable F: Append-Only Decision Log (.memlog.md)

The living audit trail of your AI-assisted design session:

- * Log of all agent interactions (Saga, Winston, etc.).
 - * Documented prompt strategies and refinement cycles.
 - * Key design debates, trade-offs encountered, and why specific paths were selected over others.
 - * Student justifications on the approvals, rejections, changes and own inputs.
-

6. Workflow Guidelines & Level 3 AI Rules

1. **Context Window Isolation:** Follow the SDD step sequence strictly. Run each major phase (wds-0-project-setup, wds-1-project-brief, bmad-spec, bmad-architecture) in a **fresh context window** or dedicated session to prevent context pollution.
 2. **Level 3 AI Co-Design:** You are the architect and director; the AI agent is your staff analyst and technical challenger.
 - Challenge generic outputs.
 - Require the agent to justify architectural constraints.
 - Perform adversarial self-review on every generated section for omissions, vagueness, or anti-patterns before accepting it into .memlog.md.
 3. **No Code Implementation:** Do **NOT** write application code, create database migrations, or configure runtime deployments in this task. This assignment terminates at the design/spec boundary.
-

7. Evaluation Rubric

Submissions will be evaluated against a 100-point rubric assessing rigor, completeness, and handoff quality:

Evaluation Dimension	Description & Quality Indicators	Weight
1. Peer Review Remediation &	Systematic resolution of Workshop 2 findings across all 8 dimensions. Transparent mapping between peer critiques	15%

Evaluation Dimension	Description & Quality Indicators	Weight
Traceability	and Spec 2 enhancements.	
2. Spec Contract Precision & Spec Law	Flawless separation of WHAT vs. HOW in SPEC.md. Every capability possesses unambiguous intent and quantifiable, binary success signals. Zero implementation leakage.	25%
3. Architecture Spine & Invariants	High-cohesion architectural boundaries, explicit state mutation rules, robust handling of GPU concurrency/reservations, clear stack choices with strong technical rationale.	25%
4. Architecture Decisions (ADRs) & Trade-offs	Quality and depth of numbered decisions (AD-1..AD-n). Serious treatment of rejected alternatives, realistic consequence analysis, and alignment with heterogeneous GPU constraints.	15%
5. Domain Realism & Hardware Context	Accurate reflection of Kubernetes GPU device plugins, heterogeneous node allocation (24GB vs 2×48GB), Kubeflow training lifecycles, and token-level LLM metering.	10%
6. AI Audit Trail & “Handoff Test”	Complete .memlog.md reflecting level-3 AI collaboration. The overall artifact bundle is clean, professional, and ready for immediate autonomous agent story implementation. Also, student justifications on the approvals, rejections, changes and own inputs.	10%

8. Submission & Format Requirements

- **Repository Format:** Markdown files placed in the exact directory hierarchy outlined in Section 4.
- **Diagrams:** All architecture and state diagrams must be written in valid, readable Mermaid syntax.
- **Language:** English throughout all artifacts.
- **Submission Deadline:** Will be announced in the course schedule.